

OFFICIAL PROJECT HANDOVER · CONFIDENTIAL

Kiddies Town ECD & Academy Management Portal

Complete System Handover Documentation — Operations, Administration & Technical Reference

This document constitutes the complete and final handover pack for the Kiddies Town ECD & Academy web portal. It covers day-to-day operation of all three portals, administrative controls, the technical architecture, database schema, API reference, security and POPIA compliance posture, and maintenance guidance for the receiving party.

DOCUMENT	DATE	STATUS	SYSTEM VER.
Handover Pack v1.0	22 August 2026	Final — For Acceptance	API v2.0.0

■ Document Control & Acceptance

Attribute	Detail
Project	Kiddies Town ECD & Academy — Web Management Portal (internal name: <code>kiddies-town-portal</code>)
Description	Full-stack early childhood development management platform serving Parents, Teachers, and the Principal/Administrator
Version	Handover Pack v1.0 · System API v2.0.0
Date of handover	22 August 2026
Classification	CONFIDENTIAL — CLIENT ONLY
Deliverables included	Complete source code, production build (<code>dist/</code>), this documentation pack, demo walkthrough video (<code>kiddies-town-walkthrough.mp4</code>), developmental guide PDF template, environment template (<code>.env.example</code>)

CONFIDENTIALITY NOTICE

This document contains system credentials, security architecture details, and references to learner data structures. It is provided solely for the named receiving party of the Kiddies Town ECD & Academy portal and must be stored securely in line with the Protection of Personal Information Act (POPIA), Act 4 of 2013.

Acceptance & Sign-Off

By signing below, the receiving party confirms receipt of the deliverables listed above, has been granted the administrative credentials set out in Section 4, and accepts the system in its documented state, including the known limitations recorded in Section 10.

PREPARED BY — SERVICE PROVIDER

Name & Signature

Date

ACCEPTED BY — KIDDIES TOWN CLIENT REPRESENTATIVE

Name & Signature

Date

Table of Contents

1	Executive Overview
2	System Capabilities Summary
3	User Guide
	3.1 Accessing the System & Signing In
	3.2 Parent Hub
	3.3 Teacher Console
	3.4 Principal Admin Center
	3.5 Online Enrolment Wizard
4	Administration & Operations Guide
5	Technical Architecture
6	Database Schema
7	API Reference
8	Environment Configuration & Deployment
9	Security & POPIA Compliance
10	Known Limitations & Handover Notes
11	Support, Maintenance & Recommendations

1 Executive Overview

The Kiddies Town ECD & Academy Portal is a custom-built, fully responsive web application that digitises the core operations of the school. It replaces ad-hoc paper registers, verbal updates, and scattered messaging with a single secure platform that serves three audiences through dedicated portals:

- **Parents** — real-time visibility of their child's attendance, weekly learning themes, progress reports, fee statements and receipts, school events with RSVP, and a direct messaging channel to staff.
- **Teachers** — daily attendance capture, milestone and progress assessment across ten developmental indicators, classroom journal publishing, theme planning, and parent communication.
- **The Principal / Administrator** — centralised oversight of enrolments, learners, staff, transport routes, medical profiles, finances, bulk parent communications, emergency broadcasts, and a full audit trail.

The platform is built on a modern, widely supported technology stack (React, TypeScript, Node.js/Express, PostgreSQL) that is inexpensive to host and straightforward for any competent web developer to maintain. Data protection is a first-class concern: access is role-restricted, passwords are cryptographically hashed, every significant write action is recorded in an audit log, and data structures were designed around South Africa's POPIA obligations.

BUSINESS VALUE AT A GLANCE

One system for enrolment → attendance → assessment → reporting → billing → communication. Parents self-serve routine questions; teachers capture evidence once; the principal sees live school-wide statistics without compiling spreadsheets.

In Scope of This Handover

- All three dashboards plus the public landing page and online enrolment wizard.
- REST API (v2) with authentication, role-based access control, validation, rate limiting, and audit logging.
- Database schema (PostgreSQL / Neon serverless) with a local JSON-file fallback mode for demonstrations.
- Dynamically generated PDF documents (parent developmental guide).
- Source code repository layout, build pipeline, deployment procedure, and operational runbook.

Out of Scope

- Payment gateway integration (payments are currently captured as ledger records/receipts; no card acquiring).
- Automated off-site backups and monitoring/alerting services (recommendations in Section 11).
- Content such as photographs, policies, and fee values — these remain editable data owned by the school.

2 System Capabilities Summary

Brand Landing Page

Premium public showcase with program hero galleries, filterable photography sliders, and an information grid covering compliance, billing buffers, milestones, and CBD shuttle pick-ups.

Parent Hub

Child attendance history, visual weekly schedules and themes, fee notices and payment receipts, teacher chat, event calendar with RSVP, and downloadable progress reports.

Teacher Console

Attendance tracker, milestone assessments across ten developmental indicators (social-emotional, fine motor, literacy, numeracy, etc.), journal editor, theme manager, and direct parent messaging.

Principal Admin Center

School-wide statistics, student directory, staff roster, calendar manager, transport route manager, medical profiles, bulk email and arrears notices, audit log viewer, and database reset utility.

Online Enrolment Wizard

Six-step validated application form: child particulars, parent details, medical profile, transport needs, POPIA consents, and document upload — feeding directly into the admin enrolment pipeline.

Dynamic PDF Generation

Server-compiled, high-fidelity PDF documents via pdftkit, including the downloadable ECD Developmental Guide for parents.

Secure Authentication

Email + password + role login with JWT access tokens (15 min) and refresh tokens (7 days), bcrypt password hashing, brute-force rate limiting, and role-matched endpoint guards.

Audit Trail

Every significant write operation is attributed to the acting user and inspectable in the Admin Center's Audit Log Viewer.

Class Groups

Learners are organised into three class groups, configurable as a database enum: ROSES GIRAFFES TIGERS

3 User Guide

3.1 Accessing the System & Signing In

- 1 Open the portal address in any modern browser (Chrome, Edge, Firefox, Safari — desktop or mobile). The landing page is public.
- 2 Select **Login** and enter your school email address and password.
- 3 Select the profile/role you are signing in as (Parent, Teacher, or Principal/Admin). The role must match the account's assigned role or sign-in is refused.
- 4 You are taken straight to your role's dashboard. Sessions expire after 15 minutes of token lifetime; the app renews automatically while in use.

PASSWORD GUIDANCE

On first production use, each seeded account holder should change their temporary password immediately (see Section 4 for the seeded credential list). Passwords are never displayed by the system to anyone, including administrators.

3.2 Parent Hub

MODULE	WHAT IT DOES
Attendance View	Daily arrival status and attendance history per enrolled child.
Weekly Themes	The current classroom theme, description and activities so parents can reinforce learning at home.
Progress Reports	Milestone scorecards across ten developmental indicators with teacher comments; released terms become visible here and can be downloaded/printed.
Payment History	Fee ledger, invoice status flags, and school reference codes with receipt numbers.
Event Calendar	School events and concerts with category filters and one-click RSVP.
Chat Portal	Direct, logged message channel between the family and the school.

3.3 Teacher Console

MODULE	WHAT IT DOES
Attendance Tracker	Capture the daily register per class group; statuses feed straight into parent views.
Milestone Tracker & Progress Editor	Grade each child across ten developmental indicators (social-emotional, fine motor, gross motor, literacy, numeracy, etc.), add comments, and release term reports to parents.
Journal Editor	Publish weekly classroom highlights visible to parents.
Theme Manager	Create/edit the weekly learning themes and activity lists shown to parents.
Teacher Chat	Two-way messaging with parents from within the console.

3.4 Principal Admin Center

MODULE	WHAT IT DOES
Overview Statistics	Live counters: enrolled learners, pending applications, tuition statistics, attendance snapshot.
Enrolment Pipeline	Review applications submitted via the wizard, track step/status, approve or decline — approval provisions the learner record.
Student Directory	All learners with class grouping, guardian links, and profile cards; create/update/delete (delete is admin-only).
Staff Roster	Teacher and staff listing.
Calendar Manager	Create and manage school events (categories: concerts, meetings, outings, closures...).
Transport Manager	Manage CBD shuttle routes and learner allocations via route modals.
Medical Profiles	View/edit sensitive medical details under restricted admin access.
Bulk Email & Arrears Notices	Send announcements to parent groups or targeted arrears reminder notices.
Audit Log Viewer	Searchable trail of who did what and when (user email, action, payload, timestamp).
Payments Ledger	Record payments against learners; receipts are numbered automatically.
Database Reset Utility	Restores demonstration seed data (see Section 4.3). <i>Use with care in production.</i>

3.5 Online Enrolment Wizard

Public-facing, no login required. Applications proceed through six validated steps:

- 1 Child Particulars** — names, date of birth, ID number, gender, home language, religion, requested grade/class.
- 2 Parent Details** — contacts, addresses, marital status, guardianship ("child lives with").
- 3 Medical Profile** — allergies, conditions, medication, doctor details.
- 4 Transport Details** — shuttle/CBD pick-up requirements.
- 5 Consents** — POPIA information-processing consents and school policy acceptance.
- 6 Document Upload** — supporting documents (birth records, immunisation cards, etc.).

Every step is validated inline before progression; incomplete submissions cannot advance. Submitted applications appear instantly in the Principal Admin Center's enrolment pipeline with a reference status.

4 Administration & Operations Guide

4.1 Seeded Accounts (Demo Credentials)

The following accounts are provisioned automatically when the system initialises. They are intended for immediate post-handover access and training. **Change these passwords before going live with real learner data** (Section 11, item 1).

ROLE	EMAIL ADDRESS	PASSWORD	ACCOUNT HOLDER
PARENT	parent@kiddiestown.co.za	parent	Sarah Mbeki
TEACHER	teacher@kiddiestown.co.za	teacher	Teacher Anne
ADMIN	admin@kiddiestown.co.za	admin	Shineon M.

IMPORTANT

These bootstrap passwords are simple by design for handover training. They are stored hashed (bcrypt), but must be replaced before the system holds real personal information. New parent accounts can be created by admins via `POST /api/admin/create-parent` or by public signup at the registration screen.

The seed also includes two demo learners (Jake & Jill Mbeki) linked to the parent account so that every screen has realistic content during orientation.

4.2 Daily Operational Notes

- **Teachers** should complete the attendance register each morning; parents see the update live.
- **Progress reports** are drafted per term and only visible to parents once marked *released*.
- **Bulk emails** require a configured mail provider key if enabled in the hosting environment; confirm delivery settings before relying on them for emergencies.
- **Emergency broadcasts** use the same bulk-email channel — test quarterly.

4.3 Resetting Demonstration Data

For open days, training, or returning the system to a clean state, the administrator can reset all tables back to seed content:

```
POST /api/admin/reset-db      # requires Admin JWT; wipes current data and re-seeds mock content
```

DESTRUCTIVE ACTION

In production with a live database this erases real records. Take a backup first (Section 11, item 2) and restrict knowledge of this capability to the principal and the technical maintainer.

4.4 Health Check

Hosting monitors can poll the unauthenticated health endpoint to confirm the service is alive:

```
GET /api/health              # returns version 2.0.0 and uptime status
```


5 Technical Architecture

5.1 Stack Overview

LAYER	TECHNOLOGY
Frontend framework	React 19 + TypeScript 5.8, React Router 7 (lazy-loaded, role-protected routes)
Styling	Tailwind CSS 4 with premium design tokens (Outfit / Inter / JetBrains Mono typography)
Client state	Zustand (auth/session store), TanStack React Query (server data)
Forms & validation	react-hook-form + Zod schemas shared conceptually with the server
Backend	Node.js + Express 4 (modular: config / controllers / middleware / routes/v1 / schemas / db / utils)
Security middleware	helmet, CORS allow-list, compression, express-rate-limit, JSON Web Tokens, bcryptjs (cost 12)
Database	PostgreSQL via Neon serverless driver + Drizzle ORM (typed queries & migrations)
Fallback store	Local JSON file (<code>data_store.json</code>) when no <code>DATABASE_URL</code> is configured
PDF engine	pdfkit (server-side document compilation)
Logging	pino (+ pino-pretty in development)
Build tooling	Vite 6 (client bundle), esbuild (server bundle to <code>dist/server.mjs</code>)
AI integration (optional)	@google/genai (Gemini) used by the pitch/demo console when <code>GEMINI_API_KEY</code> is present

5.2 Repository Layout

```
├─ server/                                # CURRENT modular backend (entry: server/index.ts → app.ts)
│   ├── config/ controllers/ middleware/ routes/v1/ schemas/ db/ utils/
│   └─ server.ts                          # LEGACY single-file backend – kept for reference only
├─ src/
│   ├── app/router.tsx                    # routes: / , /login , /enrol (public); /admin /parent /teacher (guarded)
│   ├── features/                         # landing, auth, admin, parent, teacher, enrollment pages
│   ├── components/                       # ui primitives + admin/ teacher/ parent/ enrollment/ modules
│   ├── stores/authStore.ts               # session state (localStorage keys kt_session_token, kt_logged_in_user)
│   └─ types.ts                           # shared domain types & academic scoring enums
├─ scripts/
│   ├── dev.mjs                           # dev launcher (esbuild → .tmp/server.mjs, then plain node)
│   ├── e2e-test.mjs                     # Playwright end-to-end smoke tests
│   └─ record-tour.mjs                    # records the client walkthrough video
├─ presentation/                          # handover artifacts (walkthrough video, guide PDF)
├─ dist/                                  # production build output (client + server.mjs)
└─ .env.example                           # environment variable template
```

5.3 Authentication Flow

- 1 User submits email + password + selected role to `POST /api/auth/login` . The role must match the stored account role.
- 2 Password verified against its bcrypt hash (cost factor 12). A legacy plaintext-comparison fallback exists in the login path for pre-migration accounts (flagged in Section 10).

- 3 Server issues a signed HS256 **access token** (15 min, issuer `kiddies-town`, secret `JWT_SECRET`) and a **refresh token** (7 days, secret `JWT_REFRESH_SECRET`). Both are returned in the JSON response body.
- 4 The client keeps the access token in browser localStorage and attaches it as a `Bearer` header. The refresh token is not persisted client-side; renewal uses `POST /api/auth/refresh-token`.
- 5 Each protected request passes `requireAuth()`; write endpoints additionally pass `requireRole('parent' | 'teacher' | 'admin')`. Unauthenticated visitors receive read-only guest data (public themes, events, journal only).

5.4 Request Pipeline (middleware order)

```
helmet (security headers; CSP relaxed in dev)
→ CORS allow-list (credentials, CORS_ORIGIN)
→ compression
→ body parsing (JSON & urlencoded, 10 MB limit)
→ rate limiting (general API: 100 req / 15 min · auth endpoints: 10 req / 15 min)
→ JWT bearer authentication (optional guest fallback, role 'guest')
→ RBAC role guard (per-route)
→ Zod schema validation (request bodies)
→ audit-log middleware (records writes)
→ route handlers → central error handler (always last)
```

5.5 Data Modes

- **Cloud mode** — when `DATABASE_URL` is set, the server connects to PostgreSQL (Neon), bootstraps all tables/enums on start-up, and seeds baseline content if empty. This is the production mode.
- **Local file mode** — without `DATABASE_URL`, all reads/writes map to `data_store.json`, seeded from mock data. Ideal for demos/training on a laptop; not sized for production volume. The file contains PII and is git-ignored by policy.

6 Database Schema

Schema is defined in `server/db/schema.ts` using Drizzle ORM. Eight PostgreSQL enums provide referential safety: `role`, `gender`, `class_type`, `attendance_status`, `payment_status`, `sender`, `event_category`, `enrolment_status`. Indexes exist on e-mail, status and date columns.

TABLE	PURPOSE & KEY COLUMNS
<code>kt_users</code>	Portal accounts. Email (primary key), name, bcrypt <code>password_hash</code> , <code>role</code> enum, creation timestamp.
<code>kt_learners</code>	Learner records: full names, DOB, ID number, gender, language, religion, grade/school, <code>class_type</code> , attendance status & arrival time, guardian email, enrolment approval flag, transport allocation fields.
<code>kt_parent_profile</code>	Guardian profile: contacts, physical address, marital status, "child lives with", mother/father details (JSONB), emergency contact, POPIA consent markers.
<code>kt_progress_reports</code>	Term scorecards: learner, year, term, released flag, days absent, ten-indicator scores (JSONB), teacher comments.
<code>kt_payments</code>	Fee ledger: description, date, amount, <code>payment_status</code> , receipt number, parent email, learner link.
<code>kt_chats</code>	Parent–school messages: sender enum, sender name, text, timestamp, parent email thread key.
<code>kt_weekly_themes</code>	Curriculum themes: week number (PK), title, description, activities (JSONB).
<code>kt_school_events</code>	Events: title, date, time, <code>event_category</code> , RSVP list (JSONB).
<code>kt_journal_posts</code>	Weekly classroom highlight entries published by teachers.
<code>kt_enrolments</code>	Wizard applications: five JSONB step payloads (child, parent, medical, transport, consents), current step, <code>enrolment_status</code> , date applied.
<code>kt_audit_logs</code>	Immutable activity trail: user email, action, request payload (JSONB), timestamp.

SCHEMA CHANGES

Apply changes through Drizzle Kit (`drizzle.config.ts`) rather than editing the live database directly, so that the typed ORM layer, enums, and bootstrap seeder stay in sync.

7 API Reference

All routes are mounted under both `/api/v1` and `/api` (backward compatibility). Unless marked *public*, requests require a `Bearer` access token; role requirements are noted. Bodies are JSON and validated with Zod.

7.1 Authentication

Endpoint	Access	Description
<code>POST /auth/signup</code>	Public	Register a parent/teacher/admin profile. Parent registrations are auto-provisioned with demo children, starting milestones and sample fee ledgers for instant onboarding.
<code>POST /auth/login</code>	Public (rate-limited 10/15 min)	Returns access + refresh tokens; selected role must match account role.
<code>POST /auth/refresh-token</code>	Refresh token	Exchanges a valid refresh token for a new access token.

7.2 Data Retrieval

Endpoint	Access	Description
<code>GET /data</code>	Any (guest fallback)	Combined dashboard feed filtered to the caller's role; guests receive only public themes/events/journal.
<code>GET /all-data</code>	Any	Legacy alias of <code>/data</code> .
<code>GET /health</code>	Public	Service health probe (v2.0.0).

7.3 Resource Writes

Endpoint	Roles	Description
<code>POST /learners</code>	admin · teacher · parent	Create/update learner records.
<code>DELETE /learners/:id</code>	admin	Remove a learner record.
<code>POST /parent-profile</code>	parent · admin	Update guardian profile & consents.
<code>POST /progress-reports</code>	admin · teacher	Save/release milestone scorecards.
<code>POST /payments</code>	admin · parent	Capture fee ledger entries/receipts.
<code>POST /chats</code>	all roles	Post a message to a parent-school thread.
<code>POST /events</code>	admin · teacher	Create school events.
<code>POST /themes</code>	admin · teacher	Publish weekly learning themes.
<code>POST /journal</code>	admin · teacher	Publish classroom journal posts.
<code>POST /enrolments</code>	admin · parent	Submit/update wizard applications.

7.4 Admin Operations

ENDPOINT	ROLES	DESCRIPTION
POST /admin/create-parent	admin	Provision a new parent account.
POST /admin/send-bulk-emails	admin	Announcement broadcast to parent groups.
POST /admin/send-arrears-notice	admin	Targeted fee-arrears reminders.
GET /admin/audit-logs	admin	Paged audit trail query.
POST /admin/reset-db	admin	Destructive re-seed to demonstration state (\$4.3).

7.5 Documents

ENDPOINT	ROLES	DESCRIPTION
GET /pdf/guide	Authenticated	Streams the dynamically compiled ECD developmental guide PDF (pdffkit).

8 Environment Configuration & Deployment

8.1 Environment Variables

VARIABLE	REQUIRED	PURPOSE
JWT_SECRET	REQUIRED	Signing key for access tokens. Minimum 32 characters; generate with <code>node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"</code> .
JWT_REFRESH_SECRET	REQUIRED	Signing key for refresh tokens (same generation method).
DATABASE_URL	OPTIONAL	Neon PostgreSQL connection string. Omitted ⇒ local JSON-file mode (demos only).
PORT	OPTIONAL	Listen port (default 3000).
NODE_ENV	OPTIONAL	<code>development</code> <code>production</code> <code>test</code> . Production serves the built client from <code>dist/</code> .
CORS_ORIGIN	OPTIONAL	Allowed browser origin (default <code>http://localhost:3000</code>). Set to the public domain in production.
GEMINI_API_KEY	OPTIONAL	Google Gemini key for the AI pitch/demo console.
APP_URL	OPTIONAL	Public URL of the hosted app (used in generated content).

Copy `.env.example` to `.env` and complete it. Never commit `.env`; it is excluded by `.gitignore`.

8.2 Commands

```
npm install      # install dependencies
npm run dev      # development server on port 3000 (hot reload; esbuild → .tmp/server.mjs + Vite middleware)
npm run lint     # TypeScript diagnostics (tsc --noEmit)
npm run build    # production build: Vite client bundle + esbuild server bundle (dist/server.mjs)
npm start       # run the compiled production build (serves API + built client on PORT)
npm run clean    # remove build artifacts
```

8.3 Deployment Procedure (typical)

- 1 Provision a PostgreSQL instance (e.g., Neon) and copy its connection string into `DATABASE_URL`.
- 2 Set strong `JWT_SECRET` / `JWT_REFRESH_SECRET` values and `NODE_ENV=production`, `CORS_ORIGIN=https://your-domain`.
- 3 `npm ci && npm run build` on the host (or build in CI and ship `dist/`).
- 4 `npm start` under a process manager (pm2, systemd, Docker, or platform runtime). The server bootstraps the schema automatically on first connect.
- 5 Verify `GET /api/health`, then log in with the admin account and change the seeded password.

A ready-made production bundle ships in `dist/` and can be started immediately with `npm start` (plus a completed `.env`).

9 Security & POPIA Compliance

9.1 Controls Implemented

- **Authentication & sessions** — short-lived JWT access tokens, refresh rotation endpoint, role-matched login.
- **Password storage** — bcrypt hashing (cost 12); hashes are never returned by any endpoint.
- **Authorisation** — every write endpoint declares its permitted roles; deletion of learners is admin-only; medical data sits behind admin access.
- **Input hardening** — Zod schema validation on all mutating requests; parameterised SQL via Drizzle ORM (mitigates injection).
- **Transport & headers** — helmet security headers; enable HTTPS at the hosting layer (recommended reverse proxy / platform TLS).
- **Abuse prevention** — general API limit 100 req/15 min; strict 10 req/15 min limiter on authentication endpoints.
- **Data minimisation for guests** — unauthenticated visitors see only public curriculum/event/journal content; no learner identities.
- **Auditability** — `kt_audit_logs` records user, action, payload and timestamp for significant writes.
- **PII hygiene** — `data_store.json` and `.env` are git-ignored; documents advise against committing PII.

9.2 POPIA Alignment

OBLIGATION	HOW THE PLATFORM ADDRESSES IT
Lawful processing & consent	Enrolment wizard captures explicit POPIA information-processing consents (Step 5) before an application is accepted.
Minimality	Only ECD-operational fields are collected; role-scoped feeds mean each user sees only what they need.
Security safeguards	Hashed credentials, RBAC, rate limiting, validation, audit trail (§9.1).
Children's information	Child identities, birth records, addresses and phone numbers are never publicly indexed or exposed to unauthenticated visitors.
Operator accountability	Audit logs support internal investigations and evidence of processing activities.
Data subject participation	Parents view and correct their household profile via the Parent Hub; export/deletion requests can be fulfilled by the administrator with developer assistance.

RESIDUAL RESPONSIBILITY

POPIA compliance is shared: the school remains the responsible party for lawful grounds, retention limits, and breach-notification duties. Ensure the hosting provider agreement covers data processing, and schedule periodic review of the audit log and user accounts.

10 Known Limitations & Handover Notes

#	ITEM	DETAIL / RECOMMENDATION
1	Seeded passwords are trivial	<code>parent</code> / <code>teacher</code> / <code>admin</code> defaults exist for training. Change all three before admitting real learner data.
2	Legacy plaintext comparison fallback in login	The login path retains a fallback comparison for pre-migration accounts alongside the bcrypt check. Recommend migrating any remaining accounts and removing the fallback (single-file change in the auth controller).
3	Token storage location	Access token is held in browser localStorage (<code>kt_session_token</code>), which is readable by page scripts. Acceptable for the current threat model; recommend moving to HttpOnly Secure cookies as a future hardening step.
4	JSON-file mode is demo-grade	Without <code>DATABASE_URL</code> the app persists to <code>data_store.json</code> on disk. Fine for demonstrations; always configure Postgres for production.
5	Legacy code kept for reference	Root <code>server.ts</code> (~1,400-line monolith) and <code>src/App.legacy.tsx</code> are superseded by <code>server/</code> and <code>src/features/</code> . They are excluded from normal runs (<code>npm run dev:legacy</code> only) and may be deleted after a settling-in period.
6	No payment gateway	Payments are a managed ledger only; card acquiring/EFT automation would be a new work package.
7	Email delivery depends on host configuration	Bulk-email/arrears features require provider credentials in the environment; verify before relying on them operationally.
8	Windows path quirk workaround	<code>scripts/dev.mjs</code> exists because <code>tsx</code> mis-handles the space in the folder path ("New folder") on Windows; keep using <code>npm run dev</code> rather than invoking <code>tsx</code> directly.
9	Rate limits are conservative defaults	Tune <code>express-rate-limit</code> ceilings if the school grows significantly or integrates kiosk devices.

11 Support, Maintenance & Recommendations

11.1 Routine Maintenance Calendar

CADENCE	ACTION
Daily	Glance at health endpoint / service uptime monitor.
Weekly	Review Audit Log Viewer for unexpected activity; confirm backups ran.
Monthly	Apply OS/runtime security patches; review user accounts (disable leavers); spot-check released reports.
Quarterly	Restore-test a database backup; rotate JWT secrets if policy requires; test emergency broadcast channel.
Annually	Dependency upgrade review (npm outdated / npm audit); POPIA retention clean-up of stale records.

11.2 Priority Recommendations After Handover

- 1
- Change all three seeded passwords and distribute private credentials to real staff.
- 2
- Configure automated backups (Neon point-in-time recovery or scheduled dumps) and store copies outside the server.
- 3
- Enable HTTPS end-to-end and set CORS_ORIGIN / APP_URL to the production domain.
- 4
- Remove the legacy plaintext login fallback once all accounts are confirmed hashed (item 10.2).
- 5
- Adopt uptime monitoring against /api/health with alerting to the principal's mailbox.
- 6
- Archive or delete the legacy files (server.ts , App.legacy.tsx) after one month of stable operation.

11.3 Where Things Live (Quick Reference)

NEED	LOCATION
Developer/architecture README	README.md (repository root)
Walkthrough video for staff training	presentation/kiddies-town-walkthrough.mp4
Sample developmental guide PDF	presentation/kt-ecd-developmental-guide.pdf
Environment template	.env.example
Database schema definitions	server/db/schema.ts
Seed/demo data & bootstrap accounts	server/db/bootstrap.ts
End-to-end tests	scripts/e2e-test.mjs (Playwright)

— End of handover pack. Prepared for the official transfer of the Kiddies Town ECD & Academy portal, 22 August 2026. —